Dynamic memory and valgrind

<u>The most basic explanation</u>

There are two ways to reserve memory in a C program:

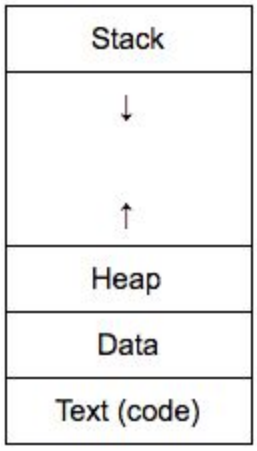
- Via **static memory allocation**
- Via **dynamic memory allocation**

To properly understand the difference between the two methods, programmers will need to review how the memory layout of a program is used (see below), but in the most basic of senses:

- Static memory allocation is determined **before** execution.
  - Concerns variables that have been defined and initialized in the code
    - int number;
    - char value;
    - double array[50];
  - These variables are <u>static</u> because they **cannot be changed** (because they are placed on the *stack*).

- Dynamic memory allocation is determined **during** execution.
  - Concerns variables that have been initialized using an allocation function;
    - int * array = malloc(sizeof(int) * 10);
    - double newArray = calloc(5*5, sizeof(double));
    - realloc(string, sizeof(char) * 60);
      *where string was a previously allocated char * variable*.
  - These variables are dynamic → they are placed "on reserve" on the heap and remain there until cleaned via the *free* command.

<u>Memory layout</u>



Stack:
- Automatic allocation
- **Static allocation**
- Variables and parameters for each function call

Heap:
- Manual (run-time) allocation
- **Dynamic allocation**

Data:
- Globals, literals, statics
- "Known at compiler time"

Text:
- Program instructions (code)

How does a program allocate memory [Run-through example]

Let's consider the following program:

```c
#include <stdio.h>
#include <stdlib.h>

int myFunction(int * array, int number)
{
    return number + array[0];
}

int * myDynamic(int * dynamicArray)
{
    int * newArray = malloc(sizeof(int) * 50);
    return newArray;
}

int main()
{
    /* Static variables */
    int array[50] = {5};
    int number = 17;

    /* Dynamic variables */
    int * dynamicArray = NULL;

    /* Using static variables */
    int myValue = myFunction(array, number);
    printf("myValue is %d\n", myValue);

    /* Using dynamic memory */
    dynamicArray = myDynamic(dynamicArray);
    if (dynamicArray == NULL)
    {
        printf("Error: Malloc failed\n");
        return -1;
    }

    free(dynamicArray);

    return 0;
}
```

1) Firstly, memory in a computer can be viewed as a large 2D grid, where each block of memory contains four important aspects:

1. Name → the variable name
2. Address → the location of the block
3. Value → the value assigned to a specific variable
4. Type → the variable type (int, double, char, float, pointers, etc.)

Computer memory can be visualized like the above grid, albeit much, MUCH, larger.

2) The reason types are important is because every variable type uses up a certain amount of bytes.

- int uses 4 bytes
- double uses 8 bytes
- char uses 1 byte

This means that each block in our grid is **sized differently.** Blocks with different types could be larger or smaller to other blocks containing different types.
Pointers are thought as indexes that point to a specific address, but not an actual block.

---

*I know this could seem intimidating or hard to understand, so instead visualize our computer memory as a **warehouse**. Our warehouse uses different sized and colored containers:*

- *Small → represents our **char** variables*
- *Medium → represents our **int** variables*
- *Large → represents our **double** variables*
- *Pointers will be index cards. So, where in place of where a container would go, a worker would find a card telling them where they could find the particular box.*

For example, *int number = 17;* would be in a block of memory where:

- The name is **number**
- The address is **1st row, 1st column***
- The value is **17**
- The type is **int**

**The address won't actually be shown as 1st row, 1st column when checking the memory address (it will show something like 0x1aec13c543). This is just to make the computer memory easy to visualize*

So I'll compile the c file, where it will go through the whole C preprocessor process (see *Objects* tutorial for more information) and come out with an executable.

At this point there exists:

- A block assigned for **number** with a value of **17**
- A continuous chain of 50 blocks for **array**. Each index is given a value of **5.**
- An index for a variable called **dynamicArray.**

---

*So our warehouse looks **similar to the above picture.***
Imagine that our boxes are put away in a first-come-first-serve system (meaning as soon as we get our block, we put it away sequentially so they come one after another).
*So, using the above program as an example, we have one red container (for number), 50 containers (for each index of array), and an index card pointing to a particular location in our warehouse where a box **may or may not** exist. **However, this particular location is reserved for small containers (int variable types) only!!***

---

3) Now, our memory is set to be used with our program. Once we run the executable, here's what happens:

A. number (with value of 17) is set on the **stack**
    a. **Stack: 1, Heap: 0**
B. array (50 blocks with a value of 5 each) is set on the **stack**
    a. **Stack: 51, Heap: 0**
C. A pointer to dynamicArray is set on the **stack. This is just the index, not the actual variable block!!**
    a. **Stack: 51, Heap: 0**
D. The function myFunction is called.
    a. **Stack: 51, Heap: 0**
E. Look at the parameters. Array is **passed-by-reference** (meaning we only reference the original block of code), but number is **passed-by-value** (so we created a new variable with the same value as number). This new number is set on the **stack**.
    a. **Stack: 52** (51 from main, 1 from myFunction), **Heap: 0**

F. Once we reach the end of our function (myFunction), all the new variables that were set on our stack only are erased.
    a. **Stack: 51** (the 1 from myFunction is erased), **Heap: 0**
G. Once we return from our function, our variable myValue is initialized. This variable is static, so it is set on the **stack.**
    a. **Stack: 52, Heap: 0**
H. The function myDynamic is called.
    a. **Stack: 52, Heap: 0**
I. Now, remember that our dynamicArray is only a pointer (an index!). We created a new variable <u>pointer</u> in myDynamic called newArray. However, this pointer is given some memory via malloc (50 blocks to be precise)! The index is placed on the **stack,** <u>but</u> the data is placed on the **heap.**
    a. **Stack: 53, Heap: 50**
J. We reach the end of our function myDynamic, but the memory that was malloc'd is reserved. We return to main, but the memory on the heap is <u>not</u> erased. The index is removed from the stack though.
    a. **Stack: 52, Heap: 50**
K. Once we return to main, dynamicArray is <u>not</u> initialized. Remember, it is only an index. It only points to the same location of memory that was reserved. No new blocks are set.
    a. **Stack: 52, Heap: 50**
L. Once we call free(), the memory <u>pointed</u> to by dynamicArray **on the heap** is freed. The index is still there, but now it points to somewhere empty (hence the index is still on the stack)
    a. **Stack: 52, Heap: 0**
M. We reach the end of main. Remember that main is technically a function, so all the variables that were set on the stack by main is erased
    a. **Stack: 0, Heap: 0**

Before we see how this would work in our warehouse example, let's define some things first:

- *Stack: Containers placed on the truck for shipment*
- *Heap: Special containers placed nearby until they need to be used*
- *Functions: Item orders. Once we complete an order (a function), the truck ships out with the stack*

So, let's begin:

A. *The workers place our number container on the **main** truck (the stack)*
B. *The workers place our array containers (50) on the **main** truck (the stack)*
C. *The workers get an index card telling them where they **might** find small container / a bunch of small containers. This index is placed on the **main** truck*
D. *A new order called myFunction is provided. A new truck for myFunction is prepared*
E. *The workers place our number container (this is a different container with a different address) placed on the **myFunction** truck*
F. *We reach the end of the **myFunction** order. The truck ships out.*

G. *Returning to the main order, the workers place our myValue container on the **main** truck.*
H. *A new order called myDynamic is provided. A new truck for myDynamic is prepared*
I. *The workers get an index card telling them where they **might** find small container / a bunch of small containers. They put the index card on the **myDynamic** truck. Using the index, they find a special order of 50 small containers (the malloc'd memory)*
J. *The end of the myDynamic order is reached. The **myDynamic** truck (only containing an index card) ships out. The special order remains in the warehouse.*
K. *Our original dynamicArray index is modified (written over) to point to the location of the special order.*
L. *The workers don't need the special order anymore, so they free it (the put the special order away)*
M. *We reach the end of our **main** order. The **main** truck ships out.*

Using malloc()

- A function found in stdlib. Used to dynamically allocate memory.
- Syntax: malloc(number of bytes)
  - malloc(sizeof(int));
    - Gives you memory that can store an int.
  - malloc(sizeof(int)*3);
    - Gives you memory for 3 ints (an array of size 3)
- Catch the returned address in a pointer for use
- It returns NULL on failure. It can fail. To avoid segfaults, ALWAYS check for NULL.

Leaking memory and free()

- When you exit a program, or return from a function, stack memory is automatically cleaned for you
- You have to manage the clean-up of heap memory on your own
- For every malloc(), you must call free
  - Failure to do so will cause memory leaks

Using valgrind to check for leaks

- A memory debugging tool that can check for memory leaks and diagnose other kinds of memory errors
- A program needs to be compiled with the **-g** flag to use valgrind
  - gcc program.c -o runMe -g
- Run valgrind with a compiled program to check for memory which has been allocated and not freed
  - valgrind ./runMe
- Valgrind can print out a lot of information about the memory usage of your program. The most important sections are the **HEAP SUMMARY** and **LEAK SUMMARY**

Extra: Using calloc()

- The same as malloc(), but initializes the variable to ‘0’
    - Malloc will assign random values initially until overwritten
- Because it dynamically allocates memory **and** initializes the memory to ‘0’, it is slower than malloc()
- The syntax is different too. Unlike malloc(), which needs the sizeof(THING)*range, calloc takes two arguments → the size of the variable, and the sizeof(THING)
    - malloc(sizeof(int)*3) is the same as calloc(3, sizeof(int))
    - malloc(sizeof(char)*5*2) is the same as calloc(5*2, sizeof(char))
- This can also fail, so always check for NULL to see if process was successful.
- Like malloc, this needs to be freed.

Extra: Using realloc()

- Resizes the memory block pointed by a pointer that was previously allocated dynamically with malloc or calloc
- Syntax: Takes two arguments; the pointer to be resized and the new size of the pointer
    - realloc(string, sizeof(char)*10)
      where previously, string was called by malloc(sizeof(char)*5)
- This can also fail, so always check for NULL to see if process was successful.